Experiments On A Robotic Car: Light Seeking, Line Following, Obstacle Avoidance, And Environmental Sensing



by

Yuwei Zhao, Haohao Qi, Yilin Zhang

Submitted for the

Final report

ROBOTICS

SCHOOL OF MATHEMATICAL AND COMPUTER SCIENCES

HERIOT-WATT UNIVERSITY

June 2025

Abstract

This project presents the design, implementation, and validation of “Bionic Lobster (Lobster)”, a multifunctional robotic vehicle inspired by crustacean morphology. Through systematic integration of optoelectronic, inertial, and environmental sensors with custom Printed Circuit Board (PCB) implementations, the platform demonstrates four autonomous capabilities: phototactic navigation using Light-Dependent Resistor (LDR) sensors achieving 29/30 evaluation score; Proportional-Integral-Derivative (PID)-controlled line tracking at 0.5 m/s via TCRT5000 infrared array; obstacle avoidance in confined spaces using HC-SR04 ultrasonic scanning with geometric clearance modeling; and context-aware mobility adaptation through BME280 environmental sensing coupled with MPU6050 inertial measurement.

The lobster-themed mechanical design, realized through Blender modeling and 3D printing, provides functional compartmentalization while enabling strategic component placement. Experimental validation confirms robust performance across all operational modes, with particular success in PID-stabilized trajectory tracking and acceleration-magnitude-based state transition (thresholds: 8.5 m/sš suspension, 10.0 m/sš ground contact). Current implementation maintains modular specialization where only one functionality operates at a time, suggesting future integration pathways through sensor fusion techniques. This work establishes a versatile platform for autonomous navigation research while demonstrating practical applications of embedded sensor systems in robotic mobility.

The whole project stores in Group 15 GitHub Repository.

Keywords: autonomous robotics, sensor fusion, PID control, environmental sensing, mechatronic design

Contents

1 Introduction **1**
- 1.1 Our Car Lobster . . . 1
- 1.2 Four Aims of the Laboratory . . . 1
- 1.3 Result Summary . . . 2

2 Experiments **3**
- 2.1 Light Seeking . . . 3
  - 2.1.1 Theory and Technical Background . . . 3
  - 2.1.2 Method . . . 4
  - 2.1.3 Results . . . 5
- 2.2 Line Following . . . 6
  - 2.2.1 Theory and Technical Background . . . 6
  - 2.2.2 Method . . . 6
  - 2.2.3 Results . . . 8
- 2.3 Obstacle Avoidance . . . 10
  - 2.3.1 Theory and Technical Background . . . 10
  - 2.3.2 Method . . . 11
  - 2.3.3 Results . . . 12
- 2.4 Environment Sensing . . . 13
  - 2.4.1 Theory and Technical Background . . . 13
  - 2.4.2 Method . . . 14
  - 2.4.3 Results . . . 15

3 Our Car Lobster **17**
- 3.1 Bodywork . . . 17
- 3.2 Construction . . . 17

4 Conclusions **20**

References **21**

Bibliography **22**

A Contribution Statement **23**

B Source Code **24**
- B.1 Lab 3: Light Seeking . . . 24
  - B.1.1 lightToward.cpp . . . 24
- B.2 Lab 4: Line Following . . . 26
  - B.2.1 dataTrans.h . . . 26
  - B.2.2 Motion_setting.h . . . 26
  - B.2.3 Motion_control.h . . . 27
  - B.2.4 main.cpp . . . 29
- B.3 Lab 7: Obstacles Avoiding . . . 30
  - B.3.1 Motion_setting.h . . . 30

B.3.2 Motion_control.h 31
B.3.3 Servo_control.h 32
B.3.4 Ultrasound_control.h 33
B.3.5 main.cpp 33
B.4 Lab 8: Environment Measurement 35
B.4.1 MotionDetect.h 35
B.4.2 MotionSetting.h 35
B.4.3 MotionControl.h 36
B.4.4 FactorDetect.h 37
B.4.5 LCDDemonstrate.h 38
B.4.6 main.cpp 39

Chapter 1

Introduction

1.1 Our Car Lobster

The main aim of this laboratory is to build our own robotic car named "**Lobster**". Its name comes from the shape, which in our opinion is a good sigh of strength and speed. Haohao Qi is responsible for our car's shape and color design, who measures its internal dimensions and builds a 3D model using Blender. Yuwei Zhao operates a 3D printer with permission from his club.

In southern China, lobsters are considered a delicious delicacy. When harvesting them in the fields, locals find them difficult to catch due to their speed. Similarly, we hope our robotic car, Lobster, will be fast and robust. More detail see chapter 3.

1.2 Four Aims of the Laboratory

We have done four contributions to it, which are recorded in sections: 2.1 (light-seeking), 2.2 (line following), 2.3 (obstacle avoidance) and 2.4 (environment sensing). Thus, Lobster could have the following functionalities:

1. Follow the light on the floor and adjust its direction toward the light source.
2. Follow a 1 to 3 cm wide black line on the floor. The line can be straight, curved, or quarter-shaped.
3. Move straight and avoid obstacles. When reaches obstacles such as walls, Lobster will turn to another direction to avoid collisions.
4. Measure temperature, humidity and atmospheric pressure, display the readings and adjust motor speed accordingly.

If Lobster has installed light seeking module, then it cannot track lines any more, and the same applies to the other two functionalities. In short, only one of these four modules works at a time.

The circuit implementations were designed using JLCPCB's EDA tool, with four custom PCB

layouts created for clear visualization of interconnections. A PCB provides reliable physical realization of circuit designs, replacing traditional wiring with etched copper traces. More information about JLCPCB here pro.lceda.cn.

1.3 Result Summary

Basic approaches of the four experiments have been successfully completed.

We used two LDRs to detect changes in light intensity. Lobster can recognize a straight white light in the front of it within **15 cm** long and **10 cm** wide.

PID control was creatively applied in line tracking. We used TCRT5000 Infrared Transducer (IR) sensor feedback to distinguish between black and non-black surfaces on the ground. Interestingly, we found that sensor feedback exhibits a linear response while in the middle of black and white, which enabled the implementation of PID control. As a result, Lobster is able to move at a speed of **0.5 m/s** in straight lines, and **0.3 m/s** in curved ones.

For obstacle avoidance, Lobster was placed in a $3 \times 3\,\mathrm{m}^2$ square place with walls around. HC-SR04 ultrasonic distance sensor was used in this experiment to measure mainly front distance. Additionally, SG90 servo motor was applied to change HC-SR04's direction to avoid a side crash. Lobster passes a **1-minute** test in is square place without any collisions.

In the eighth laboratory experiment, we used a BME280 sensor to measure humidity, atmospheric pressure, and temperature. The collected data is displayed on a small screen mounted on Lobster's back. Based on these measurements, Lobster can determine whether it is on the ground or in the air. Accordingly, its wheel rotation speed is adjusted based on the detected state by a MPU6050 sensor.

Chapter 2

Experiments

2.1 Light Seeking

2.1.1 Theory and Technical Background

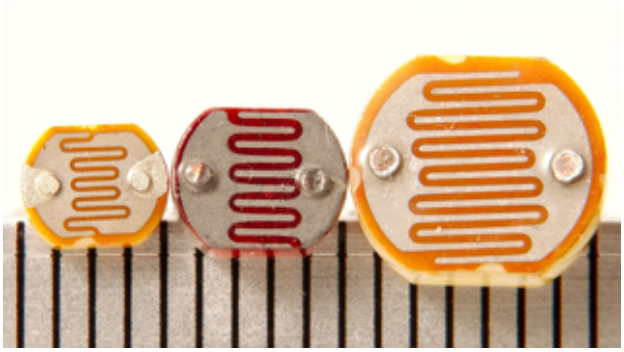


Figure 2.1: Photoresistors.Resource: makeabilitylab.github.io/physcomp

The operating principle of LDRs is rooted in the photoelectric properties of semiconductor compounds. When exposed to light, materials like cadmium sulfide (CdS) undergo photon absorption that generates electron-hole pairs within their crystalline structure. As illumination intensifies, this photogeneration effect produces a greater number of charge carriers, thereby enhancing the material's conductivity. Consequently, LDRs exhibit an inverse relationship between incident light intensity and electrical resistance-a characteristic that makes them valuable for light-sensing applications. This nonlinear response proves particularly sensitive in the 10–100 lux range, where a 20% increase in light intensity causes approximately 60% resistance drop.

LDRs in our system demonstrate a measurable resistance range from 1 kΩ under direct sunlight to 1.5 MΩ in complete darkness, as recorded by our multimeter.

We use two DC gearbox motors (TT motors) to control Lobster's movement, which typically requires Pulse-Width Modulation (PWM) for speed regulation.

The three key parameters of PWM control are frequency, duty cycle, and resolution, defined as follows:

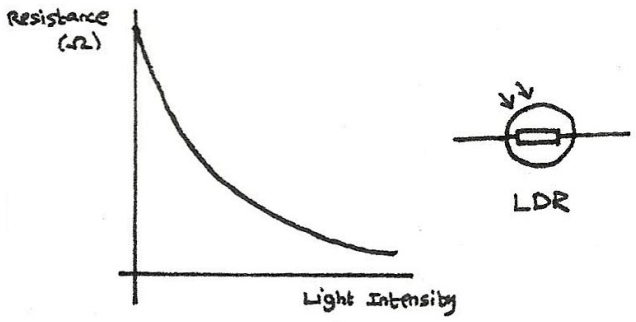


Figure 2.2: LDRs' Intensity-Resistance curves.Resource: revise.im

$$\begin{cases} \text{Freq} = \frac{1}{T_S} \\ \text{Duty} = \frac{T_{\text{ON}}}{T_S} \\ \text{Reso} = \frac{1}{T_S} \end{cases} \tag{2.1}$$

where *Freq*, *Duty*, and *Reso* represent the frequency, duty cycle, and resolution of the PWM signal, T_S is the total period of one PWM cycle, and T_ON is the high-level (active) time within the cycle.

The duty cycle is the most critical parameter. For example, a 50% duty cycle corresponds to approximately half of the motor's full speed, while a 100% duty cycle drives the motor at maximum speed.

These characteristics directly influence Lobster's light-seeking behavior.

2.1.2 Method

With *Arduino*'s help, glspwm control can be simply done by setting motor's voltage. In package, it is 0 to 255.

Two LDRs were mounted on the left and right front sides of Lobster, each returning a sensor reading between 0 and 1023. Then, the LDR values were scaled down by a factor of 4 to match the motor input range. Lobsters steering direction was determined by the difference in illumination between the two LDRs: if the left-side reading was lower than the right, the right motors voltage was reduced (and vice versa), causing Lobster to turn toward the brighter side. Additionally, if both sensors detected insufficient light, set to over 1019, the motors would stop entirely, preventing movement in darkness. It can be described in pseudocode as follows:

```
if(Left < 1019 || Right < 1019) {
  if(Gap >= expect_Gap) {
    if(Left < Right) {
      turnLeft();
    } else {
      turnRight();
    }
  } else {
    goStraight();
  }
} else {
  stop();
}
```

The phototaxis module's circuit implementation is shown in figure 2.3, designed using JLCPCB's EDA suite.

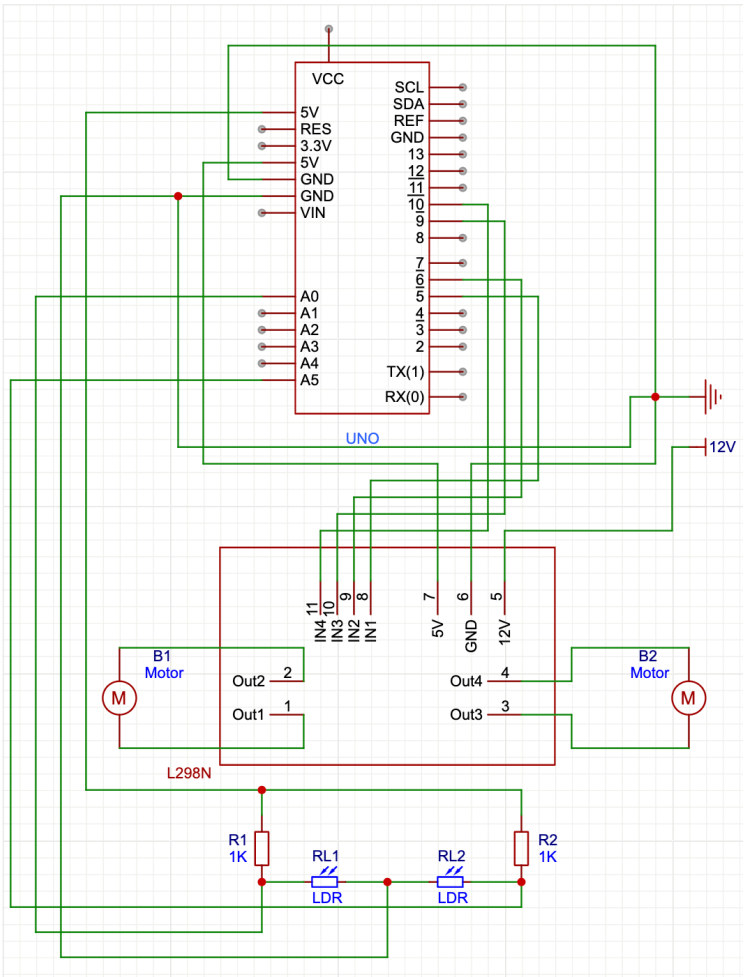


Figure 2.3: A PCB design for the light-seeking experiment, including a symmetrical placement of two LDR sensors (LR1, LR2), and two motor drivers connections with PWM control traces.

2.1.3 Results

In the end, we managed to set straight speed at 120. Lobster passed the test twice. On the first attempt, its turning speed was set too fast, which caused uncontrollable swaying, and our teacher

was not satisfied with that performance. On the second attempt, it performed very well, scoring 29 out of 30, losing 1 point for requiring two attempts.

2.2 Line Following

2.2.1 Theory and Technical Background

Shown in figure 2.4, *TCRT5000* serves as the cornerstone in this experiment. This reflective optoelectronic device integrates both an infrared emitter (950nm photodiode) and receiver (NPN phototransistor) in a compact 4-pin package, distinguished by its blue emitter and black receiver components. [1]

During operation, the photodiode transmits modulated infrared signals while the phototransistor detects reflected radiation. The sensor's daylight-blocking optical filter ensures reliable operation under ambient lighting conditions.

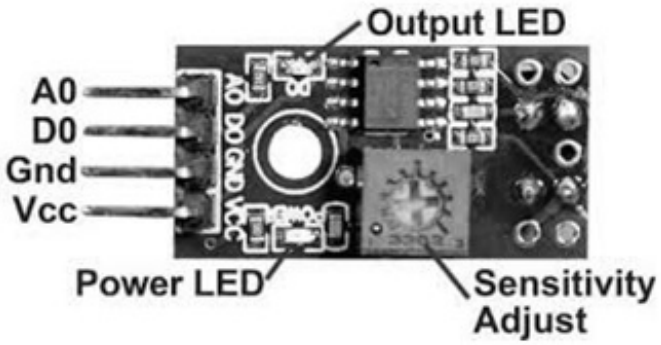


Figure 2.4: A TCRT5000 IR Sensor Module mainly includes parts like TCRT5000 IR sensor, power LED, reflective sensor, adjustment of sensitivity, and output LED

2.2.2 Method

The IR sensor voltage coming back was measured as 4.15 ± 0.1 V on black lines, and 0.15 ± 0.1 V on white ground. We observed a linear transition region that in the middle of black and white surfaces, enabling analog position detection suitable for PID control.

The linear voltage gradient permits error quantification for PID control. PID combines proportional (P), integral (I) and derivative (D) components. To implement PID control, there should be a reference value (setpoint) as the control target, standing for adjustment goal. Continuous

measurement of the process variable to get the most recent value and then calculate the *error* between the setpoint and this value.

Proportional control (P) applies a correction proportional to the current error. Simply proportional adjustment is the quickest and easiest way, which is just a proportional coefficient (Kp) times error, described as equation (2.2).

$$P = Kp \cdot error \tag{2.2}$$

Integral control (I) is similar; it accounts for accumulated past errors. Its difference is that each time the integral coefficient (Ki) times the total sum of all errors calculated in the history, described as equation (2.3).

$$I = Ki \cdot \int_0^t error \cdot dt \tag{2.3}$$

And derivative control (D), as its name suggests, is a derivative coefficient (Kd) times the deviation between this error and the last error. It responds to the error rate of change, described as equation (2.4) where dt refers to the time offset.

$$D = Kd \cdot \frac{error}{dt} \tag{2.4}$$

The total PID control is all three adjustments added together. All equations above are described mathematically, but are challenging for computers to understand. In coding environment, they need to be changed *discretely*, which, in total, can be described as equation (2.5), where $e(k)$= position error related to time t. In car situations, PD control is better than full PID control because integral term accumulates historical error causes overshoot.

$$u(k) = K_p e(k) + K_i \sum_{\tau=0}^{k} e(\tau) + K_d(e(k) - e(k-1)) \tag{2.5}$$

With only PID control, Lobster is still easy to veere off-track. We also applied a not-found-line function to overcome this issue. The function keeps recalling the last direction of deviation, and once Lobster loses its direction, it keeps rotating toward the memorized direction until any sensor detects a black line.

On Lobster, three TCRT5000 IR sensors are mounted in parallel with a 1.2 cm inter-sensor spacing. The voltage response of a single sensor exhibits a piecewise-linear characteristic, combining two distinct linear segments. Therefore, we construct a voltage-position graph figure 2.5 to analyze the sensor behavior.

The Left sensor's voltage minus the right yields a short linear segment around the origin o. The middle sensor's data verifies the peak values detected by the left/right sensors. Once the middle sensor cannot see black lines, we mirror the outermost sensor data to artificially extend the linear region.

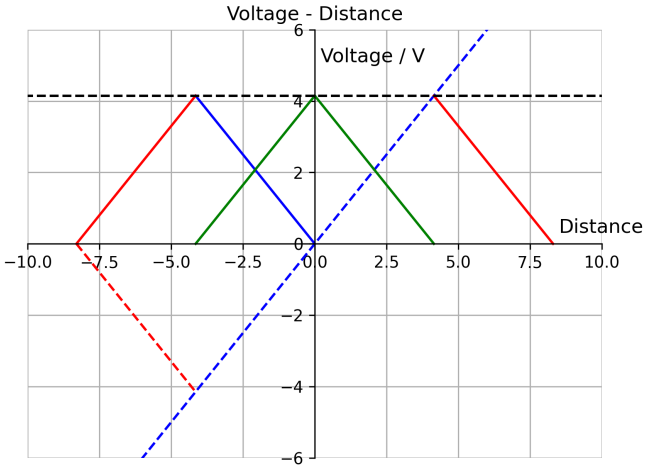


Figure 2.5: Voltage-position relationship of three TCRT5000 IR sensors. Dashed lines show the mirrored extension for linearity enhancement.

The phototaxis module's circuit implementation is shown in figure 2.6, designed using JLCPCB's EDA suite.

2.2.3 Results

Lobster successfully completed the line-following test with 100% accuracy, as demonstrated in figure 2.7. The robot maintained a tracking speed of 0.3 m/s on curved paths and 0.5 m/s on straight segments.

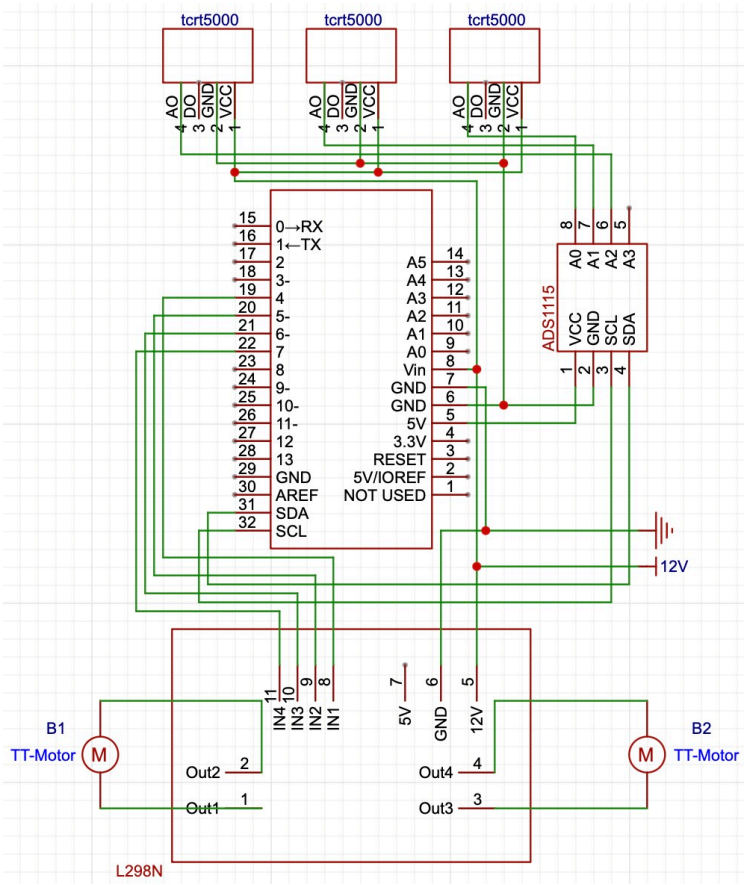


Figure 2.6: A PCB design for the light-seeking experiment, mainly including three TCRT5000 IR sensors.

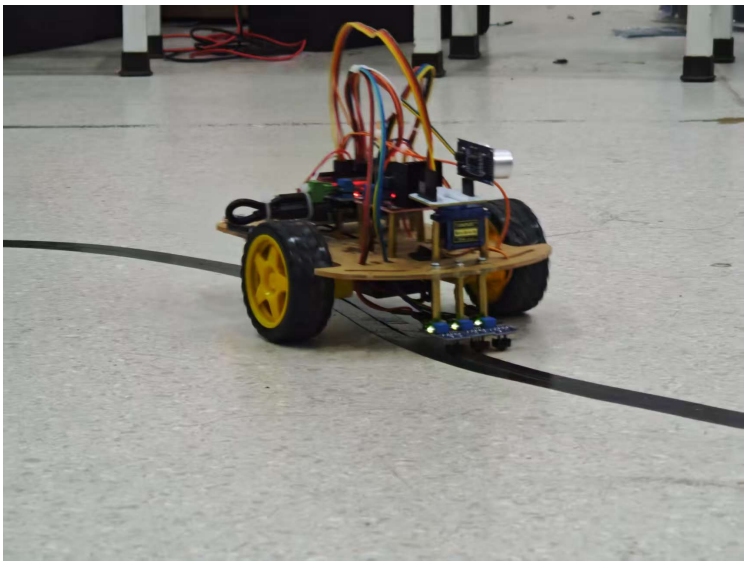


Figure 2.7: Lobster autonomously tracking a 3-cm-wide black line during the test.

2.3 Obstacle Avoidance

2.3.1 Theory and Technical Background

In this experiment, *HC-SR04* and *SG90* are the two main sensors, showed in figure 2.8.

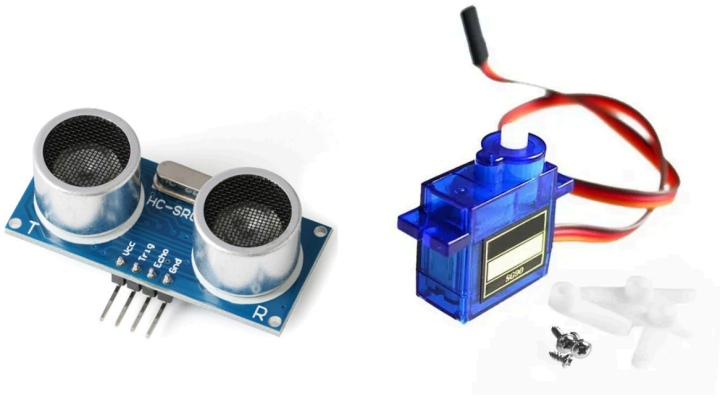


Figure 2.8: HC-SR04 ultrasonic sensor and SG90 servo motor diagram; pins of HC-SR04 are arranged from left to right as: *VCC*, *Trig*, *Echo* and *GND*.

The HC-SR04 ultrasonic sensor mainly consists of a transmitter and a receiver. This sensor has a measurement range of 2 cm to 400 cm with an accuracy of 0.3 cm. [2] Two pins are particularly important:

- The *Trig pin* serves as a trigger. Only a single high-level pulse needs to be applied to this pin during each measurement cycle.
- The *Echo pin* outputs a high-level signal when the HC-SR04 emits ultrasonic waves. This signal remains high until the sensor receives the reflected echo.

The HC-SR04 transmitter emits ultrasonic waves in one direction at a time. When these waves encounter an obstacle, they are reflected back and detected by the receiver. The distance between the HC-SR04 and the obstacle can be calculated by measuring the time difference between the emission and reception of the ultrasonic wave, combined with the speed of sound, as described in equation (2.6):

$$d = \frac{v \cdot \Delta t}{2} \tag{2.6}$$

where d is the distance, v is the speed of the ultrasonic wave, and Δt is the time difference.

During this process, the Echo pin's signal changes as follows:

- High level when the ultrasonic wave is emitted;
- Low level when the reflected wave is received.

The SG90 servo is a angle servo driver, suitable for control systems that require continuous angle changes and the ability to maintain a specific angle.

2.3.2 Method

We designed a `carDrive` function for obstacle avoidance using a finite state machine. It processes sensor inputs and executes sequenced maneuvers when obstacles are detected. The complete logic of this function is shown in figure 2.9.

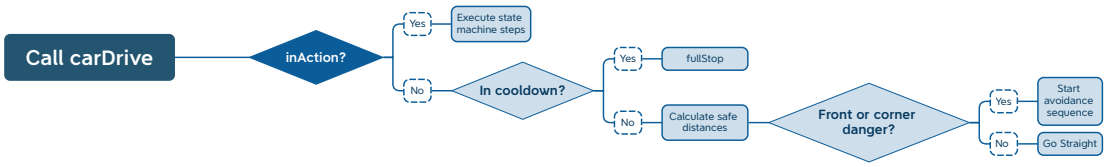


Figure 2.9: Flowchart depicting the logical structure of the `carDrive` function.

Normal Operation

In practice, Lobster may encounter obstacles from multiple directions, primarily from the left, front, and right. We mounted an SG90 servo motor on Lobster's front, with an HC-SR04 ultrasonic sensor attached. This allows the sensor to rotate and scan a wider area.

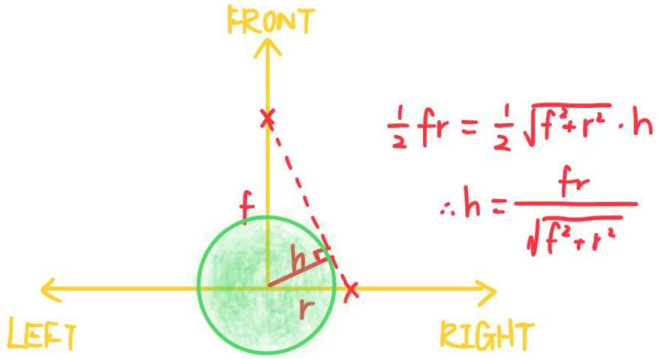


Figure 2.10: A rectangular Coordinate of the collision risk assessment model, shows fused sensor coverage (shaded).

To ensure comprehensive 180° obstacle detection, we implemented a sensor fusion method combining frontal ultrasonic ranging (40 cm threshold) and lateral infrared proximity sensing (25 cm threshold). figure 2.10 demonstrates the resulting safety envelope.

The SG90 servo rotates in 90-degree increments, performing a scanning motion at each position. Three distance measurements are obtained by: Front distance directly triggers avoidance maneuvers; Left and right distances are processed using equation (2.7):

$$\text{sideSafeDist} = \frac{\text{sideDist} \cdot \text{frontDist}}{\sqrt{\text{sideDist}^2 + \text{frontDist}^2}} \tag{2.7}$$

where: *sideSafeDist* is the effective clearance distance, *sideDist* represents left or right sensor readings, and *frontDist* is the front sensor reading

The vehicle moves straight when all clearance distances are safe. If any calculated distance falls below the threshold, the system transitions to avoidance mode.

Avoidance Maneuvers

`STEP_BACK`: First, reverse to create maneuvering space. Then check if sufficient clearance exists for a right turn (our designated avoidance direction). If adequate space exists, transition to `STEP_TURN_RIGHT_90`. Otherwise, proceed to `STEP_BACK_EXTRA`.

`STEP_BACK_EXTRA`: Perform additional reverse movement. Then transition to `STEP_TURN_RIGHT_90`.

`STEP_TURN_RIGHT_90`: Execute 90-degree right turn. Then proceed to `STEP_DONE`.

`STEP_DONE`: Complete full stop and restart the control loop.

The phototaxis module's circuit implementation is shown in figure 2.11, designed using JLCPCB's EDA suite.

2.3.3 Results

The safety thresholds were empirically determined through iterative testing. The side clearance distance was set to 25 cm to account for gradual distance reduction during lateral approach, while the frontal clearance used a more conservative 40 cm threshold to ensure adequate reaction time for head-on obstacles. For turning maneuvers, a minimum 40 cm clearance was required to execute 90ř right turns safely.

Motion timing parameters were carefully calibrated, with each reverse segment lasting 100 ms and right turns completing in 300 ms, optimized for the gearbox motor's response characteristics. In final testing within a 3 m $\times$ 3 m arena, the Lobster demonstrated consistent performance.

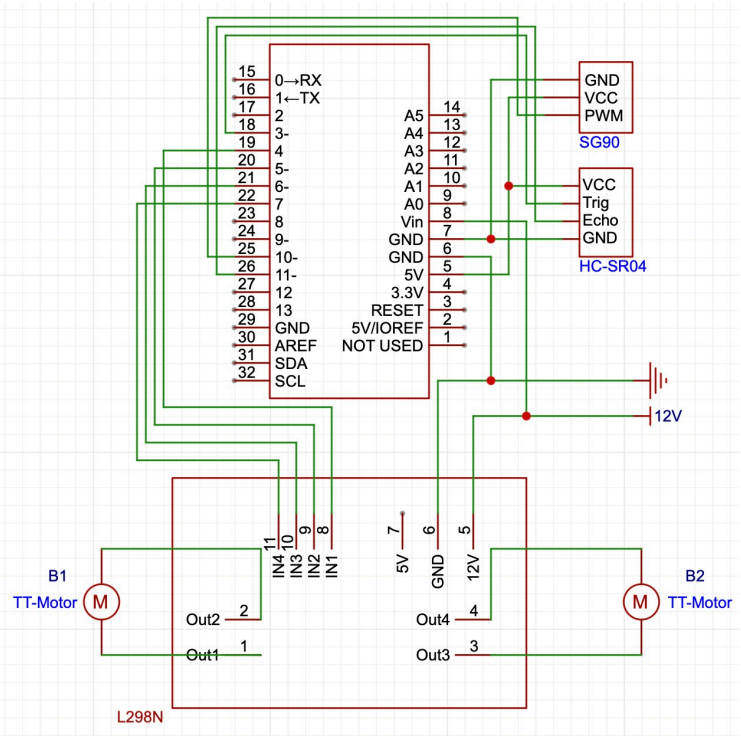


Figure 2.11: A PCB design for the light-seeking experiment, mainly includes an HC-SR04 ultrasonic sensor and a SG90 servo.

2.4 Environment Sensing

2.4.1 Theory and Technical Background

In some situations, a robotics car needs to measure humidity, atmospheric pressure, and temperature. Additionally, it needs to change its speed due to a single direction's sudden linear acceleration. Therefore, a BME280 sensor is used to overcome environment sensing problem, and an MPU6050 sensor is used to measure linear acceleration. The two sensors are as figure 2.12 shows.

A **BME280** is a temperature humidity and pressure sensor, which reads barometric pressure, temperature, and humidity. Because pressure changes with altitude, you can also estimate altitude. The BME280 sensor uses Inter-Integrated Circuit (I^2C) or Serial Peripheral Interface (SPI) communication protocol to exchange data with a microcontroller. More details here: randomnerdtutorials.com.

The **MPU-6050** is a six-axis MotionTracking device that integrates a 3-axis gyroscope and a 3-axis accelerometer on a single chip [3]. It also supports both I^2C and SPI communication interfaces, with a built-in Digital Motion Processor (DMP) for advanced motion processing tasks

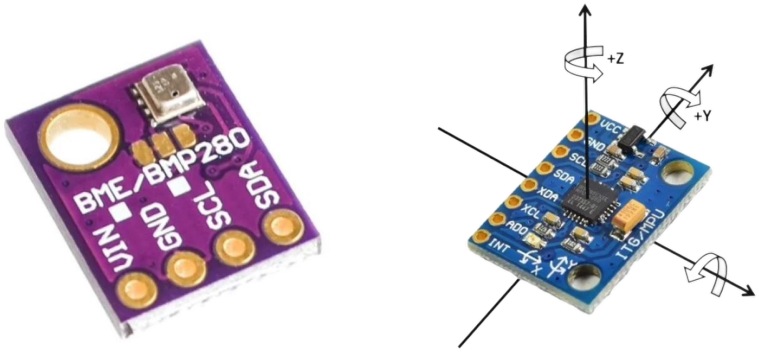


Figure 2.12: BME280 and MPU 6050 Figures.
Resource: BME280 from www.circuits-diy.com and MPU6050 from git-code.com/gh_mirrors/mpu6050/mpu6050

such as sensor fusion and gesture recognition. The device operates over a wide voltage range (2.375 V to 3.46 V), making it suitable for various embedded and wearable applications.

I^2C (Inter-Integrated Circuit), originally developed by Philips Semiconductor in 1982, is a widely used synchronous serial communication protocol designed for short-distance communication between integrated circuits [4]. It employs two bidirectional open-drain lines: SDA (Serial Data Line) and SCL (Serial Clock Line), both pulled high via external resistors. The protocol supports multiple masters and slaves on the same bus, with each slave device having a unique address. Communication begins with a START condition and ends with a STOP condition, allowing data to be transferred in packets of 8 bits, most significant bit (MSB) first. I^2C supports various speed modes including Standard Mode (100 kbps), Fast Mode (400 kbps), and High-Speed Mode (up to 3.4 Mbps), making it suitable for connecting microcontrollers with low-speed peripherals such as sensors and ADCs.

2.4.2 Method

The I^2C protocol enables synchronous data acquisition from multiple sensors through hardware addressing. Our implementation connects two sensors: BME280 at address `0x76` and MPU6050 at address `0x68`.

Acceleration data from the MPU6050 sensor is processed using the magnitude calculation:

$$a_{\text{mag}} = \sqrt{a_x^2 + a_y^2 + a_z^2} \tag{2.8}$$

where a_x, a_y, a_z represent accelerations along orthogonal axes. This composite acceleration metric provides orientation-independent motion detection, ensuring reliable state identification during arbitrary device rotations (roll/pitch/yaw). As confirmed by the implementation team, this approach prevents unintended wheel stoppage when the vehicle is lifted and subjected to spatial reorientation.

The state detection system employs threshold-based validation with temporal debouncing. Suspension state is triggered when the acceleration magnitude persistently satisfies $a_{\text{mag}} \leq 8.5\,\text{m/s}^2$ (approximately 0.87g) for over 250ms, indicating reduced gravitational influence. Ground contact is recognized when $a_{\text{mag}} \geq 10.0\,\text{m/s}^2$ (approximately 1.02g) maintains for the same duration, corresponding to normal gravitational loading.

Control logic activates propulsion when suspended to maintain wheel momentum:

$$\text{Control Output} = \begin{cases} \texttt{goStraight}(\textit{reverse}, \textit{baseSpeed}) & a_{\text{mag}} \leq 8.5\,\text{m/s}^2 \\ \texttt{fullStop}() & a_{\text{mag}} \geq 10.0\,\text{m/s}^2 \end{cases} \tag{2.9}$$

with parameters configured at $\textit{baseSpeed} = 150$ and $\textit{reverse} = \textit{false}$. This design prevents motor stall during aerial manipulation while ensuring immediate stoppage upon surface contact.

The phototaxis module's circuit implementation is shown in figure 2.13, designed using JLCPCB's EDA suite.

2.4.3 Results

The integrated sensor system demonstrated robust performance throughout experimental trials. Environmental parameters including temperature, humidity, and atmospheric pressure were successfully acquired via the BME280 sensor and displayed through dual-output channels: real-time serial monitoring on the host computer and sequential scrolling updates on a small display mounted on Lobster. This redundant visualization scheme ensured immediate parameter validation during motion operations without requiring external monitoring equipment.

Lobster successfully finished the test twice. In both trials, Lobster achieved vertical acceleration exceeding over 1g during launch phases, transitioning to suspension states at trajectory apogees. During these suspension periods, deliberate triaxial rotations were imposed through manual manipulation: roll $(\pm 90^\circ)$, pitch $(\pm 45^\circ)$, and yaw (360continuous rotation). Throughout all rotational maneuvers, wheel propulsion maintained consistent operation at the configured base

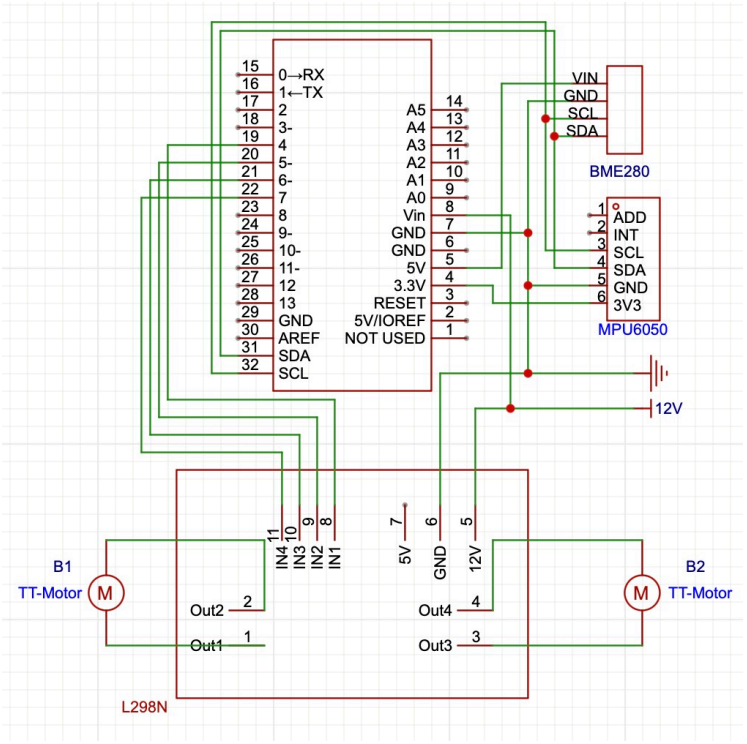


Figure 2.13: A PCB design for the light-seeking experiment, mainly includes a BME280 sensor and a MPU6050 sensor.

speed of 150 PWM units, with zero instances of unintended motor cutoff.

Chapter 3

Our Car Lobster

3.1 Bodywork

Our robot embodies a distinctive lobster **theme** throughout its design. Collaborating with co-author HaoHao Qi, we developed a custom 3D model using Blender (Fig. 3.1), which was subsequently fabricated via 3D printing. The printed shell underwent post-processing with oil-based paints, resulting in a predominantly orange exterior with black accents on the roof section.

The lobster-inspired design comprises three key anatomical segments: cephalothorax (head), pereon (body), and pleon (tail). This thematic partitioning not only enhances visual recognition but also facilitates functional component integration. The final physical implementation is shown in figure 3.2.

The exterior shell effectively conceals "goose" wires within internal cavities while maintaining external sensor accessibility. All critical sensors remain exposed to ensure unimpeded environmental interaction.

A distinctive **lighting feature** was implemented by our group, and first appeared in the preliminary experimentation. Dual rear-mounted LEDs provide turning indication: the left illuminates during left turns, while the right activates during right maneuvers. This visual signaling system enhances operational visibility in low-light conditions, and becomes Lobster's special characteristic.

3.2 Construction

The physical implementation demonstrates meticulous attention to construction standards, achieving full compliance with the requirement specification. All components are strategically positioned according to functional priorities, with the power system (primarily battery packs) housed in the undercarriage compartment, emulating modern electric vehicle architecture for optimal weight distribution. Control systems occupy the upper chassis, creating a clear separation

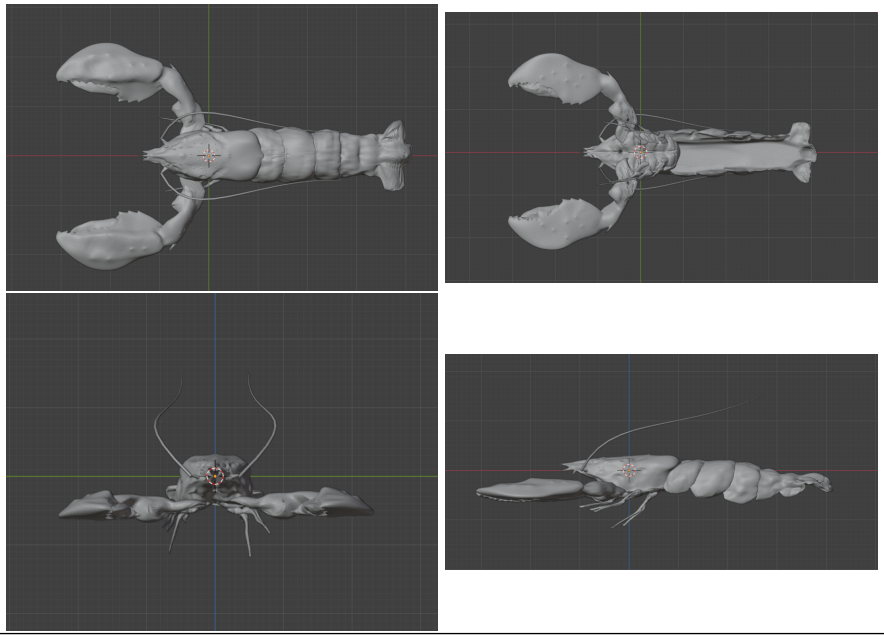


Figure 3.1: The four figures are the top, the bottom, the front and the side view. Initial Blender modeling of the lobster-themed robot. The quad-view presentation demonstrates comprehensive design considerations for component placement and aesthetic integration.

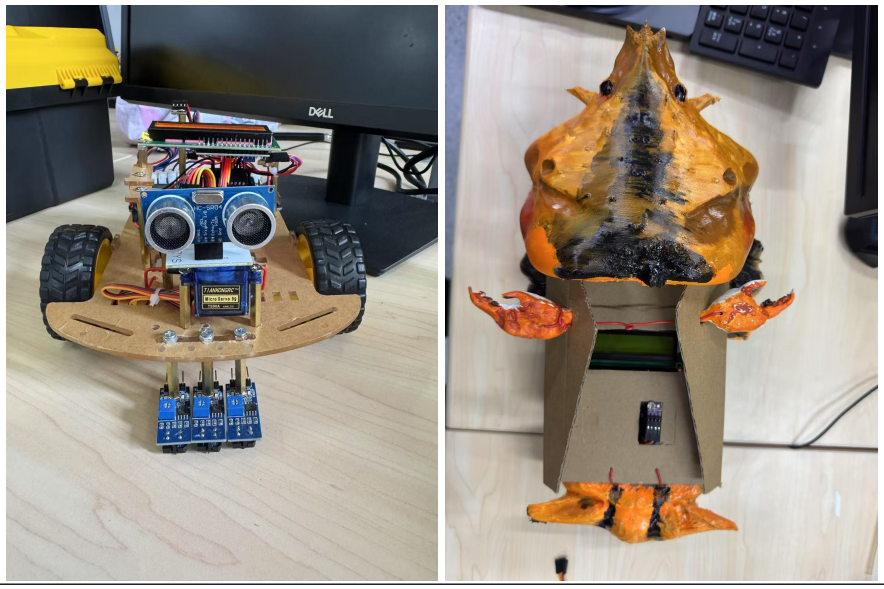


Figure 3.2: These two figures are the frontal implementation and the side perspective. Fabricated lobster robot showing: (1) Sensor integration strategy with exposed environmental interfaces, (2) Thematic color application with orange dominant and black secondary scheme, (3) Functional segmentation preserving mobility.

between power and logic subsystems.

Infrared sensors are precisely mounted at the ventral frontal position, oriented perpendicular to the ground surface for accurate line detection during phototaxis experiments. The ultrasonic sensor module is positioned about 5 cm above the anterior chassis, providing unobstructed forward obstacle detection.

Wiring implementation adheres to rigorous color-coding standards: all *Vcc* connections utilize red wire, while *ground* lines uniformly employ black insulation. Signal lines follow functional grouping with orange, yellow, and blue for different sensors.

Mechanical security exceeds basic requirements through dedicated design interventions. Custom mounting points were cut into the chassis to accommodate sensor clusters at orthogonal alignments, using human hands. The ADS1115 ADC module is secured with vibration-damping silicone mounts. All wiring harnesses are constrained at 5 cm intervals using nylon cable ties.

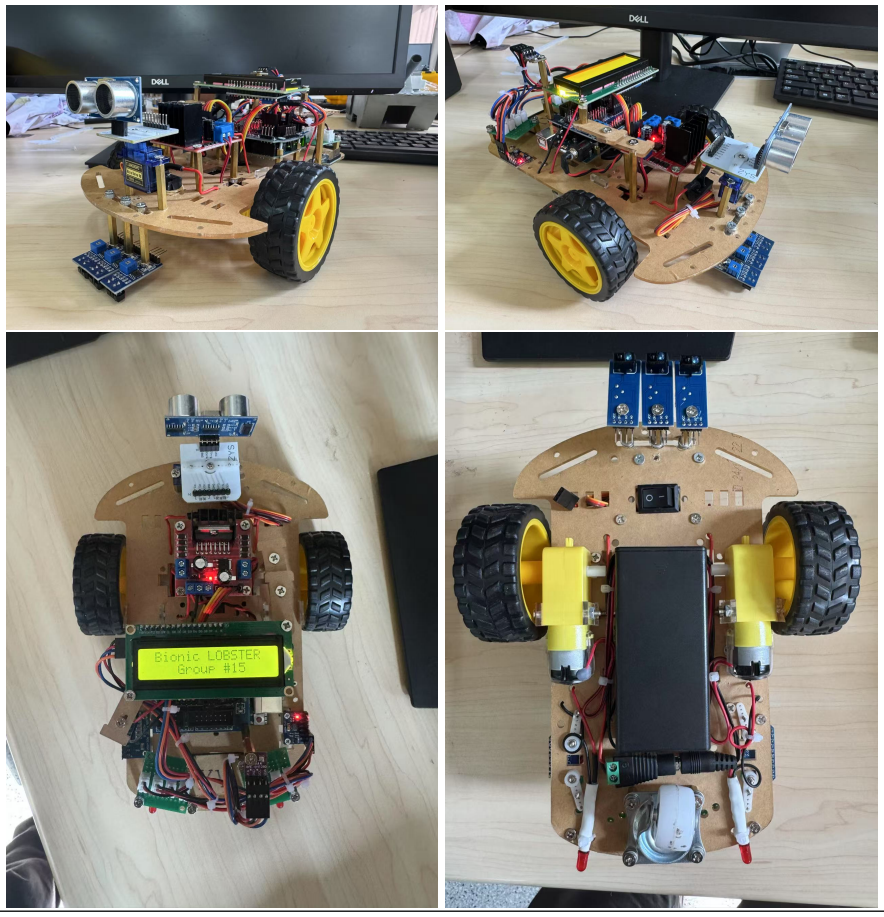


Figure 3.3: Four orthogonal perspectives (left/right/top/bottom) document the robot's compliant construction: sensor symmetry, color-coded wiring, component security and polished PCB edges.

Chapter 4

Conclusions

This laboratory successfully achieved its primary objective of developing Lobster, a multifunctional robotic vehicle inspired by the agility of lobsters. Through systematic implementation of four core functionalitieslight seeking, line following, obstacle avoidance, and environmental sensingLobster demonstrates robust performance across diverse operational scenarios.

In **phototaxis** experiments, Lobster exhibited precise light source localization within a $15 \times 10\,\text{cm}^2$ detection zone using paired LDR sensors, achieving a near-perfect evaluation score of 29/30 after motion parameter optimization. The **line tracking** module implemented innovative PID control based on TCRT5000 infrared sensors' linear transition characteristics, enabling stable navigation at $0.5\,\text{m/s}$ on straight paths and $0.3\,\text{m/s}$ on curves.

For **spatial awareness**, the integrated HC-SR04/SG90 system permitted reliable collision avoidance in confined $3 \times 3\,\text{m}^2$ environments through geometrically optimized clearance calculations. **Environmental adaptation** was realized via BME280 atmospheric sensing and MPU6050 inertial measurement, where acceleration magnitude thresholds ($\leq 8.5\,\text{m/s}^2$ for suspension; $\geq 10.0\,\text{m/s}^2$ for ground contact) enabled context-aware motor control during aerial manipulations.

The lobster-inspired mechanical **design**, fabricated through Blender modeling and 3D printing, provided both aesthetic coherence and functional compartmentalization. Strategic component placement ensured optimal weight distribution while exposed sensor interfaces maintained environmental interaction fidelity. Custom PCB implementations for each module enhanced circuit reliability and reduced wiring complexity.

While all primary objectives were met, functional modularity currently limits simultaneous operation of multiple capabilities. Future iterations could explore sensor fusion techniques to enable concurrent operation modes, potentially enhancing Lobster's autonomy in complex environments.

References

[1] *TCRT5000(L) Reflective Optical Sensor with Transistor Output*. Document Number: 83760, Revision A4. Vishay Telefunken. 2000.

[2] *HC-SR04 Ultrasonic Sensor Module User Guide*. Version 1.0. HandsOn Technology. 2021. URL: http://www.handsontec.com.

[3] *MPU-6000 and MPU-6050 Product Specification*. Document Number: PS-MPU-6000A-00, Revision 3.4. InvenSense, Inc. 2013. URL: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf.

[4] *A Basic Guide to I^2C*. Application Report SBAA565, November 2022. Texas Instruments. 2022. URL: https://www.ti.com/lit/an/sbaa565/sbaa565.pdf.

Bibliography

UKHeliBob. *Using millis() for timing. A beginners guide*. 2017. URL: https://forum.arduino.cc/t/using-millis-for-timing-a-beginners-guide/483573/1 (visited on 06/2025).

arduino_uno_guy. *I2C Liquid Crystal Displays*. Nov. 2019. URL: https://projecthub.arduino.cc/arduino_uno_guy/i2c-liquid-crystal-displays-5eb615 (visited on 06/2025).

DeepLift. *How To Make Line Follower Using Three IR Sensor | Arduino Line Following Car*. Jan. 2021. URL: https://deeplift.blogspot.com/2021/01/how-to-make-line-follower-using-three-irsensor.html (visited on 06/2025).

forthe24. *MPU6050 - Integrate euler rates to pitch,roll,yaw angle*. Oct. 2022. URL: https://forum.arduino.cc/t/mpu6050-integrate-euler-rates-to-pitch-roll-yaw-angle/1039796 (visited on 06/2025).

Appendix A

Contribution Statement

The development of Lobster represents a collaborative effort with distinct yet complementary contributions from all team members. **Yuwei Zhao** spearheaded the experimental implementation and coding development while overseeing the physical construction process, including component integration and wiring optimization. As the primary architect of the lobster-inspired design, **Haohao Qi** created the detailed 3D models using Blender software and managed the 3D printing workflow, additionally documenting key dimensional measurements throughout the fabrication process. **Yilin Zhang** contributed significantly to the experimental design phase, particularly in developing control logic architectures, while taking primary responsibility for technical documentation and report composition.

All members participated in the final bodywork assembly phase following the 3D printing completion, with Yuwei and Yilin assisting in sensor mounting and aesthetic finishing. This collective approach extended to data collection activities, where team members rotated measurement duties during experimental validation sessions to ensure consistent observation standards. The modular PCB implementations benefited from cross-verification, with each member validating at least two circuit designs beyond their primary assignments.

Appendix B

Source Code

B.1 Lab 3: Light Seeking

B.1.1 lightToward.cpp

```
/* 4PIN-MODE */
int in1Pin = 5;
int in2Pin = 6;
int in3Pin = 9;
int in4Pin = 10;

// tail lights
int leftLight = 2;
int rightLight = 8;

// speed setting
int base_Speed = 120;
int decelerated_Speed = 110;
int accelerated_Speed = 170;

// fluctuating rate
float leftComp = 1.00;
float rightComp = 1.00;

// ldr offset
int left_offset = 10;
int right_offset = -20;
int expect_Gap = left_offset - right_offset;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);
  pinMode(leftLight, OUTPUT);
  pinMode(rightLight, OUTPUT);

  // Initialize
  analogWrite(in1Pin, 0);
  analogWrite(in4Pin, 0);
  digitalWrite(in2Pin, LOW);
  digitalWrite(in3Pin, LOW);
  digitalWrite(leftLight, LOW);
  digitalWrite(rightLight, LOW);
}

void loop() {
  // put your main code here, to run repeatedly:
  boolean reverse = 0;

  int sensorValue_LEFT = analogRead(A5);
```

```
  int sensorValue_RIGHT = analogRead(A0);
  int GAP = abs((sensorValue_LEFT + left_offset) - (sensorValue_RIGHT +
  right_offset));

  Serial.print("LEFT: ");
  Serial.print(sensorValue_LEFT);
  Serial.print("\t RIGHT: ");
  Serial.print(sensorValue_RIGHT);
  Serial.print("\t GAP: ");
  Serial.println(GAP);

  Turn(reverse, sensorValue_LEFT, sensorValue_RIGHT, GAP);

  delay(10); // read arguments every 10ms
}

// Left Wheel CONTROL
void setLeft(boolean reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in2Pin, reverse);
  analogWrite(in1Pin, speed);
}

// Right Wheel CONTROL
void setRight(boolean reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in3Pin, !reverse);
  analogWrite(in4Pin, 255 - speed);
}

// car Turn CONTROL - LIGHT Toward
void Turn(boolean reverse, int Left, int Right, int Gap) {
   // If light exists
  if(Left < 1019 || Right < 1019) {
    if(Gap >= expect_Gap) {
      if(Left < Right) { // Turn LEFT
        setLeft(reverse, decelerated_Speed * leftComp);
        setRight(reverse, accelerated_Speed * rightComp);
        digitalWrite(rightLight, LOW);
        digitalWrite(leftLight, HIGH);
      }
      if(Right < Left) { // Turn RIGHT
        setLeft(reverse, accelerated_Speed * leftComp);
        setRight(reverse, decelerated_Speed * rightComp);
        digitalWrite(leftLight, LOW);
        digitalWrite(rightLight, HIGH);
      }
    } else { // Go FORWARD
      setLeft(reverse, base_Speed * leftComp);
      setRight(reverse, base_Speed * rightComp);
      digitalWrite(leftLight , HIGH);
      digitalWrite(rightLight, HIGH);
    }
  } else { // STOP
    setLeft(reverse, 0);
    setRight(reverse, 0);
    digitalWrite(leftLight, LOW);
    digitalWrite(rightLight, LOW);
  }
}
```

B.2 Lab 4: Line Following

B.2.1 dataTrans.h

```
#ifndef DataTransfer_H
#define DataTransfer_H

#include <ADS1115_WE.h>
#include <Wire.h>

extern ADS1115_WE adc;

struct IRStatus {
  float left;
  float middle;
  float right;
};

float readChannel(ADS1115_MUX channel) {
  adc.setCompareChannels(channel);
  return adc.getResult_V();
}

IRStatus statusDetect() {
  IRStatus status;
  status.left = readChannel(ADS1115_COMP_0_GND);
  status.middle = readChannel(ADS1115_COMP_1_GND);
  status.right = readChannel(ADS1115_COMP_2_GND);

  Serial.print("Left: ");
  Serial.print(status.left);
  Serial.print(" V; Middle: ");
  Serial.print(status.middle);
  Serial.print(" V; Right: ");
  Serial.print(status.right);
  Serial.println(" V");

  return status;
}

#endif
```

B.2.2 Motion_setting.h

```
#ifndef MotionSetting_h
#define MotionSetting_h

#define leftComp 0.99
#define rightComp 1.00

/* 4pin - mode */
int in1Pin = 4;
int in2Pin = 5;
int in3Pin = 7;
int in4Pin = 6;

// tail lights
int leftLightPin = 2;
int rightLightPin = 8;

// Left Wheel CONTROl
void setLeft(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
```

```
  digitalWrite(in1Pin, reverse);
  analogWrite(in2Pin, speed);
  digitalWrite(leftLightPin, reverse ? LOW: HIGH);
}

// Right Wheel CONTROL
void setRight(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in3Pin, reverse);
  analogWrite(in4Pin, speed);
  digitalWrite(rightLightPin, reverse ? LOW: HIGH);
}

// STOP
void fullStop() {
  digitalWrite(in1Pin, LOW);
  analogWrite(in2Pin, 0);
  digitalWrite(in3Pin, LOW);
  analogWrite(in4Pin, 0);
  digitalWrite(leftLightPin, LOW);
  digitalWrite(rightLightPin, LOW);
}

// Go STRAIGHT
void goStraight(bool reverse, int leftSpeed, int rightSpeed) {
  setLeft(reverse, leftSpeed * leftComp);
  setRight(reverse, rightSpeed * rightComp);
}

// Turn LEFT
void turnLeft(bool leftReverse, bool rightReverse, int leftSpeed, int
rightSpeed) {
  setLeft(leftReverse, leftSpeed * leftComp);
  setRight(rightReverse, rightSpeed * rightComp);
}

// Turn RIGHT
void turnRight(bool leftReverse, bool rightReverse, int leftSpeed, int
rightSpeed) {
  setLeft(leftReverse, leftSpeed * leftComp);
  setRight(rightReverse, rightSpeed * rightComp);
}

#endif
```

B.2.3 Motion_control.h

```
#ifndef MotionControl_H
#define MotionControl_H

#include "dataTrans.h"

#define THRESHOLD 1.0

float cal_error(IRStatus);
float calculate_pid(float);
void motor_control(float);
void lostLineHandling(float);

const int baseSpeed = 105;
const float max_pid = 60.0;

float Kp = 5.6;
float Ki = 0;
float Kd = 5;
```

```
19
20 int lostCount = 0;
21
22 float last_valid_error = 0;
23
24 void carDrive() {
25   IRStatus status = statusDetect();
26
27   float error = cal_error(status);
28
29   if(error == 666) {
30     lostCount++;
31     if(lostCount >= 10) { // 10
32       lostLineHandling(last_valid_error);
33       lostCount = 0;
34     }
35   } else {
36     lostCount = 0;
37     float pid_val = calculate_pid(error);
38     motor_control(pid_val);
39   }
40
41   Serial.print("Error: ");
42   Serial.println(error);
43 }
44
45 void lostLineHandling(float last_error) {
46   unsigned long startTime = millis();
47
48   if (last_error < 0) {
49     turnLeft(false, true, 55, 115);
50   } else {
51     turnRight(true, false, 115, 55);
52   }
53
54   while (millis() - startTime < 100) {
55     IRStatus current = statusDetect();
56
57     if (current.middle > THRESHOLD || current.left > THRESHOLD ||
     current.right > THRESHOLD) {
58       fullStop();
59       return;
60     }
61   }
62
63   fullStop();
64 }
65
66 float cal_error(IRStatus status) {
67   float error = status.left - status.right;
68   if (status.middle <= THRESHOLD) {
69     if (status.left > THRESHOLD) {
70       error = 9 - status.left;
71     } else if (status.right > THRESHOLD) {
72       error = status.right - 9;
73     } else {
74       error = 666;
75     }
76   }
77   return error;
78 }
79
80 float calculate_pid(float error) {
81   static float I = 0;
82   static float previous_error = 0;
83
84   if (error != 666) {
```

```
    last_valid_error = error;
  }

  float P = error;
  I += error;
  I = constrain(I, -25, 25);
  float D = error - previous_error;

  float PID_value = (Kp * P) + (Ki * I) + (Kd * D);

  previous_error = error;

  return constrain(PID_value, -max_pid, max_pid);
}

void motor_control(float PID_value) {
  int left_motor_speed = baseSpeed - PID_value;
  int right_motor_speed = baseSpeed + PID_value;

  left_motor_speed = constrain(left_motor_speed, 50, 200);
  right_motor_speed = constrain(right_motor_speed, 50, 200);

  goStraight(false, left_motor_speed, right_motor_speed);
}

#endif
```

B.2.4 main.cpp

```
#include "motion_setting.h"
#include "motion_control.h"
#include "dataTrans.h"

#define I2C_ADDRESS 0x48

ADS1115_WE adc = ADS1115_WE(I2C_ADDRESS);

void setup() {
  Wire.begin();
  Serial.begin(9600);

  if(!adc.init()) {
    Serial.println("ERROR: ADS1115 connected failed!");
  }

  adc.setConvRate(ADS1115_860_SPS);
  adc.setVoltageRange_mV(ADS1115_RANGE_6144);
  adc.setMeasureMode(ADS1115_CONTINUOUS);

  // Wheel Driver
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);
  // Light Driver
  pinMode(leftLightPin, OUTPUT);
  pinMode(rightLightPin, OUTPUT);
}

void loop() {
  carDrive();
}
```

B.3 Lab 7: Obstacles Avoiding

B.3.1 Motion_setting.h

```
#ifndef MotionSetting_h
#define MotionSetting_h

/* 4pin - mode */
int in1Pin = 4;
int in2Pin = 5;
int in3Pin = 7;
int in4Pin = 6;

// tail lights
int leftLightPin = 2;
int rightLightPin = 8;

float leftComp = 1.00;
float rightComp = 1.00;

// Left Wheel CONTROl
void setLeft(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in1Pin, reverse);
  analogWrite(in2Pin, speed);
  digitalWrite(leftLightPin, reverse ? LOW: HIGH);
}

// Right Wheel CONTROL
void setRight(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in3Pin, reverse);
  analogWrite(in4Pin, speed);
  digitalWrite(rightLightPin, reverse ? LOW: HIGH);
}

// STOP
void fullStop() {
  digitalWrite(in1Pin, LOW);
  analogWrite(in2Pin, 0);
  digitalWrite(in3Pin, LOW);
  analogWrite(in4Pin, 0);
  digitalWrite(leftLightPin, LOW);
  digitalWrite(rightLightPin, LOW);
}

// Go STRAIGHT
void goStraight(bool reverse, int speed) {
  setLeft(reverse, speed);
  setRight(reverse, speed);
}

// Go BACK
void goBack(bool reverse, int speed) {
  setLeft(!reverse, 255 - speed * leftComp);
  setRight(!reverse, 255 - speed * rightComp);
}

// Turn LEFT
void turnLeft(bool reverse, int speed) {
  setLeft(!reverse, 255 - speed * leftComp);
  setRight(reverse, speed * rightComp);
}

// Turn RIGHT
```

```
void turnRight(bool reverse, int speed) {
  setLeft(reverse, speed * leftComp);
  setRight(!reverse, 255 - speed * rightComp);
}

#endif
```

B.3.2 Motion_control.h

```
#ifndef MotionControl_h
#define MotionControl_h

#define front_border_VALUE 40
#define side_border_VALUE 40
#define corner_safe_VALUE 25

int baseSpeed = 200;
int turnSpeed = 100;
int deceleratedSpeed = 110;

bool reverse = false;

enum ActionStep {
  STEP_IDLE,
  STEP_BACK,
  STEP_BACK_EXTRA,
  STEP_TURN_RIGHT_90,
  STEP_DONE
};

ActionStep currentStep = STEP_IDLE;
unsigned long actionStartTime = 0;
unsigned long lastActionEndTime = 0;
bool inAction = false;
bool recentlyAvoidedCorner = false;

void carDrive(float leftDist, float frontDist, float rightDist) {
  if (!inAction) {
    if(recentlyAvoidedCorner && millis() - lastActionEndTime < 800) {
      fullStop();
      return;
    } else {
      recentlyAvoidedCorner = false;
    }
  }

  if (inAction) {
    unsigned long now = millis();
    switch (currentStep) {
      case STEP_BACK:
        if (now - actionStartTime < 100) {
          goBack(reverse, deceleratedSpeed);
        } else {
          if (rightDist <= side_border_VALUE) {
            currentStep = STEP_BACK_EXTRA;
          } else {
            currentStep = STEP_TURN_RIGHT_90;
          }
          actionStartTime = now;
        }
        return;

      case STEP_BACK_EXTRA:
        if (now - actionStartTime < 100) {
          goBack(reverse, deceleratedSpeed);
```

```
57          } else {
58            currentStep = STEP_TURN_RIGHT_90;
59            actionStartTime = now;
60          }
61          return;
62
63        case STEP_TURN_RIGHT_90:
64          if (now - actionStartTime < 300) {
65            turnRight(reverse, turnSpeed);
66          } else {
67            currentStep = STEP_DONE;
68            actionStartTime = now;
69          }
70          return;
71
72        case STEP_DONE:
73          fullStop();
74          inAction = false;
75          currentStep = STEP_IDLE;
76          lastActionEndTime = millis();
77          recentlyAvoidedCorner = true;
78          return;
79
80        default:
81          fullStop();
82          inAction = false;
83          currentStep = STEP_IDLE;
84          return;
85      }
86    }
87
88    float leftSafeDist = (leftDist * frontDist) / sqrt(pow(leftDist, 2) +
    pow(frontDist, 2));
89    float rightSafeDist = (rightDist * frontDist) / sqrt(pow(rightDist, 2) +
    pow(frontDist, 2));
90
91    fullStop();
92
93    if (leftSafeDist <= corner_safe_VALUE || rightSafeDist <=
    corner_safe_VALUE) {
94      currentStep = STEP_BACK;
95      inAction = true;
96      actionStartTime = millis();
97      return;
98    }
99
100   if (frontDist <= front_border_VALUE) {
101     currentStep = STEP_BACK;
102     inAction = true;
103     actionStartTime = millis();
104     return;
105   }
106
107   goStraight(reverse, baseSpeed);
108 }
109
110 #endif
```

B.3.3 Servo_control.h

```
1 #ifndef EngineControl_h
2 #define EngineControl_h
3
4 #include <Servo.h>
5
```

```
#define SERVO_INTERVAL 150

Servo MyServo;

int angles[] = {0, 90, 180, 90};
int curr = 0;
int lastAngle = angles[0];

unsigned long lastServoTime = 0;

int servoRotate() {
  unsigned long currentServoTime = millis();

  if(currentServoTime - lastServoTime >= SERVO_INTERVAL) {
    curr = (curr + 1) % 4;
    int angle = angles[curr];
    MyServo.write(angle);

    lastServoTime = currentServoTime;
    lastAngle = angle;
  }

  return lastAngle;
}

#endif
```

B.3.4 Ultrasound_control.h

```
#ifndef Ultrasound_h
#define Ultrasound_h

int trigPin = 3;
int echoPin = 11;

float duration;

float distanceDetect() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH);
  float distance = (duration * 0.0343) / 2;

  return distance;
}

#endif
```

B.3.5 main.cpp

```
#include "motion_setting.h"
#include "motion_control.h"
#include "servo_control.h"
#include "ultrasound_control.h"

#define ANGLE_INTERVAL 30

float leftDist = 0;
float frontDist = 0;
```

```
float rightDist = 0;

unsigned long lastAngleTime = 0;

bool isWaiting = false;

void setup() {
  Serial.begin(9600);
  // Wheel Driver
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);
  // Light Driver
  pinMode(leftLightPin, OUTPUT);
  pinMode(rightLightPin, OUTPUT);
  // Engine Driver
  MyServo.attach(10);
  // Ultrasound Driver
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {
  int angle = servoRotate();
  int distance = distanceDetect();

  unsigned long currentAngleTime = millis();

  if(!isWaiting) {
    switch(angle) {
      case 0:
        leftDist = distance;
        Serial.print("Left: ");
        Serial.println(leftDist);
        isWaiting = true;
        lastAngleTime = currentAngleTime;
        break;
      case 90:
        frontDist = distanceDetect();
        Serial.print("Front: ");
        Serial.println(frontDist);
        isWaiting = true;
        lastAngleTime = currentAngleTime;
        break;
      case 180:
        rightDist = distance;
        Serial.print("Right: ");
        Serial.println(rightDist);
        isWaiting = true;
        lastAngleTime = currentAngleTime;
        break;
    }
    carDrive(leftDist, frontDist, rightDist);
  } else if(currentAngleTime - lastAngleTime >= ANGLE_INTERVAL) {
    isWaiting = false;
  }
}
```

B.4 Lab 8: Environment Measurement

B.4.1 MotionDetect.h

```
#ifndef MotionDetect_H
#define MotionDetect_H

#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <math.h>

extern Adafruit_MPU6050 mpu;

struct variables {
  float x;
  float y;
  float z;
};

extern variables accData;
extern variables gyroData;

void getVariables() {
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  accData = { a.acceleration.x - 0.40, a.acceleration.y + 0.16,
  a.acceleration.z + 0.560 };
  gyroData = { g.gyro.x + 0.02, g.gyro.y - 0.06, g.gyro.z + 0.02 };
}

void mpuPrint() {
  getVariables();

  Serial.println("------------- MPU6050  -------------");

  Serial.println("| ----- Accelerations (m/s^2) ---- |");
  Serial.print("|    x: "); Serial.print(accData.x);
  Serial.print(", y: "); Serial.print(accData.y);
  Serial.print(", z: "); Serial.print(accData.z);
  Serial.println("    |");

  Serial.println("| --------- Gyro (rad/s) --------- |");
  Serial.print("|    x: "); Serial.print(gyroData.x);
  Serial.print(", y: "); Serial.print(gyroData.y);
  Serial.print(", z: "); Serial.print(gyroData.z);
  Serial.println("   |");

  Serial.println("-------------------------------------");
}

#endif
```

B.4.2 MotionSetting.h

```
#ifndef MotionSetting_h
#define MotionSetting_h

#define leftComp 0.99
#define rightComp 1.00

/* 4pin - mode */
int in1Pin = 4;
```

```
int in2Pin = 5;
int in3Pin = 7;
int in4Pin = 6;

// tail lights
int leftLightPin = 2;
int rightLightPin = 8;

// Left Wheel CONTROl
void setLeft(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in1Pin, reverse);
  analogWrite(in2Pin, speed);
  digitalWrite(leftLightPin, reverse ? LOW: HIGH);
}

// Right Wheel CONTROL
void setRight(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in3Pin, reverse);
  analogWrite(in4Pin, speed);
  digitalWrite(rightLightPin, reverse ? LOW: HIGH);
}

// STOP
void fullStop() {
  digitalWrite(in1Pin, LOW);
  analogWrite(in2Pin, 0);
  digitalWrite(in3Pin, LOW);
  analogWrite(in4Pin, 0);
  digitalWrite(leftLightPin, LOW);
  digitalWrite(rightLightPin, LOW);
}

// Go STRAIGHT
void goStraight(bool reverse, int speed) {
  setLeft(reverse, speed * leftComp);
  setRight(reverse, speed * rightComp);
}

#endif
```

B.4.3 MotionControl.h

```
#ifndef MotionControl_H
#define MotionControl_H

#include "MotionSetting.h"

bool reverse = false;
int baseSpeed = 150;

bool isLifted = false;

const float LIFT_THRESHOLD = 8.5f;
const float GROUND_THRESHOLD = 10.0f;
const unsigned long DEBOUNCE_TIME = 250;

void carDrive() {
  getVariables();
  static unsigned long lastStateChangeTime = 0;
  static bool stableState = false;

  float acc = sqrt(pow(accData.x, 2) + pow(accData.y, 2) + pow(accData.z,
  2));
```

```

  bool stateChanged = false;
  unsigned long currentTime = millis();

  if (stableState) {
    if (acc >= GROUND_THRESHOLD) {
      if (lastStateChangeTime == 0) {
        lastStateChangeTime = currentTime;
      } else if (currentTime - lastStateChangeTime >= DEBOUNCE_TIME) {
        stableState = false;
        isLifted = false;
        stateChanged = true;
        lastStateChangeTime = 0;
      }
    } else {
      lastStateChangeTime = 0;
    }
  } else {
    if (acc <= LIFT_THRESHOLD) {
      if (lastStateChangeTime == 0) {
        lastStateChangeTime = currentTime;
      }
      else if (currentTime - lastStateChangeTime >= DEBOUNCE_TIME) {
        stableState = true;
        isLifted = true;
        stateChanged = true;
        lastStateChangeTime = 0;
      }
    } else {
      lastStateChangeTime = 0;
    }
  }

  if(isLifted) {
    goStraight(reverse, baseSpeed);
  } else {
    fullStop();
  }
}

#endif
```

B.4.4 FactorDetect.h

```
#ifndef FactorDetect_H
#define FactorDetect_H

#include <Adafruit_BME280.h>
#include <Adafruit_Sensor.h>

extern Adafruit_BME280 bme;

float getTemperature() {
  return bme.readTemperature();
}

float getHumidity() {
  return bme.readHumidity();
}

float getPressure() {
  return bme.readPressure() / 100.0F;
}

void getBMEData(float &temp, float &hum, float &pres) {
```

```
  temp = getTemperature();
  hum = getHumidity();
  pres = getPressure();
}

void bmePrint() {
  float temp, hum, pres;
  getBMEData(temp, hum, pres);

  Serial.println("----------------------------- BME280
  ------------------------------");

  Serial.print("| Temprature: "); Serial.print(temp); Serial.print("");
  Serial.print(", Humidity: "); Serial.print(hum); Serial.print("%");
  Serial.print(", Air pressure: "); Serial.print(pres); Serial.println("hPa
  |");


  Serial.println("-------------------------------------------------------------------------");
}

#endif
```

B.4.5 LCDDemonstrate.h

```
#ifndef LCDDemonstrate_H
#define LCDDemonstrate_H

#include <LiquidCrystal_I2C.h>
#include "FactorDetect.h"
#include "MotionDetect.h"

extern LiquidCrystal_I2C lcd;

void basicDemonstrate() {
  lcd.setCursor(1, 0);
  lcd.print("Bionic LOBSTER");
  lcd.setCursor(4, 1);
  lcd.print("Group #15");
}

void bmeDemonstrate() {
  static unsigned long lastScrollTime = 0;
  static int scrollIndex = 0;

  float temp, hum, pres;
  getBMEData(temp, hum, pres);

  String fullText = "T:" + String(temp, 2) + "C "
                  + "H:" + String(hum, 2) + "% "
                  + "P:" + String(pres, 2) + "hPa ";

  const int windowWidth = 16;

  fullText += "    " + fullText;

  unsigned long currentTime = millis();
  if (currentTime - lastScrollTime > 500) {
    lcd.setCursor(0, 0);
    lcd.print(fullText.substring(scrollIndex, scrollIndex + windowWidth));
    scrollIndex++;
    if (scrollIndex > fullText.length() - windowWidth) {
      scrollIndex = 0;
    }
    lastScrollTime = currentTime;
```

```
  }
}

void mpuDemonstrate() {
  static unsigned long lastScrollTime = 0;
  static int scrollIndex = 0;

  getVariables();

  String fullText = String("acc: ") + "x:" + String(accData.x, 2) + " "
                  + "y:" + String(accData.y, 2) + " "
                  + "z:" + String(accData.z, 2) + " "
                  + String("gyro: ") + "x: " + String(gyroData.x, 2) + " "
                  + "y:" + String(gyroData.y, 2) + " "
                  + "z: " + String(gyroData.z, 2) + " ";

  const int windowWidth = 16;

  fullText += "    " + fullText;

  unsigned long currentTime = millis();
  if (currentTime - lastScrollTime > 500) {
    lcd.setCursor(0, 1);
    lcd.print(fullText.substring(scrollIndex, scrollIndex + windowWidth));
    scrollIndex++;
    if (scrollIndex > fullText.length() - windowWidth) {
      scrollIndex = 0;
    }
    lastScrollTime = currentTime;
  }
}

#endif
```

B.4.6 main.cpp

```
#include "FactorDetect.h"
#include "LCDDemonstrate.h"
#include "MotionDetect.h"
#include "MotionSetting.h"
#include "MotionControl.h"

#include <Wire.h>

Adafruit_BME280 bme;
Adafruit_MPU6050 mpu;
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  Wire.begin();
  Serial.begin(115200);
  while(!Serial);

  delay(1000);

  bme.begin(0x76);
  mpu.begin(0x68, &Wire);
  lcd.init();
  lcd.backlight();

  // Wheel Driver
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);
```

```
30    // Light Driver
31    pinMode(leftLightPin, OUTPUT);
32    pinMode(rightLightPin, OUTPUT);
33  }
34
35  variables accData;
36  variables gyroData;
37
38  unsigned long lastPrintTime = 0;
39
40  void loop() {
41    carDrive();
42
43    static bool basicShown = false;
44    if (!basicShown) {
45      basicDemonstrate();
46      basicShown = true;
47      delay(2500);
48      lcd.clear();
49    }
50
51    bmeDemonstrate();
52    mpuDemonstrate();
53
54    static unsigned long lastPrintTime = 0;
55    unsigned long currentTime = millis();
56    if(currentTime - lastPrintTime >= 1000) {
57      bmePrint();
58      mpuPrint();
59      Serial.println();
60
61      lastPrintTime = currentTime;
62    }
63  }
```